mlkem-native

A unified, fast and verified ML-KEM library

Hanno Becker[1], Rod Chapman[1], John Harrison[1], Matthias Kannwischer[2], Dusan Kostic[1]

1: Amazon Web Services
2: Chelpis Quantum Corp

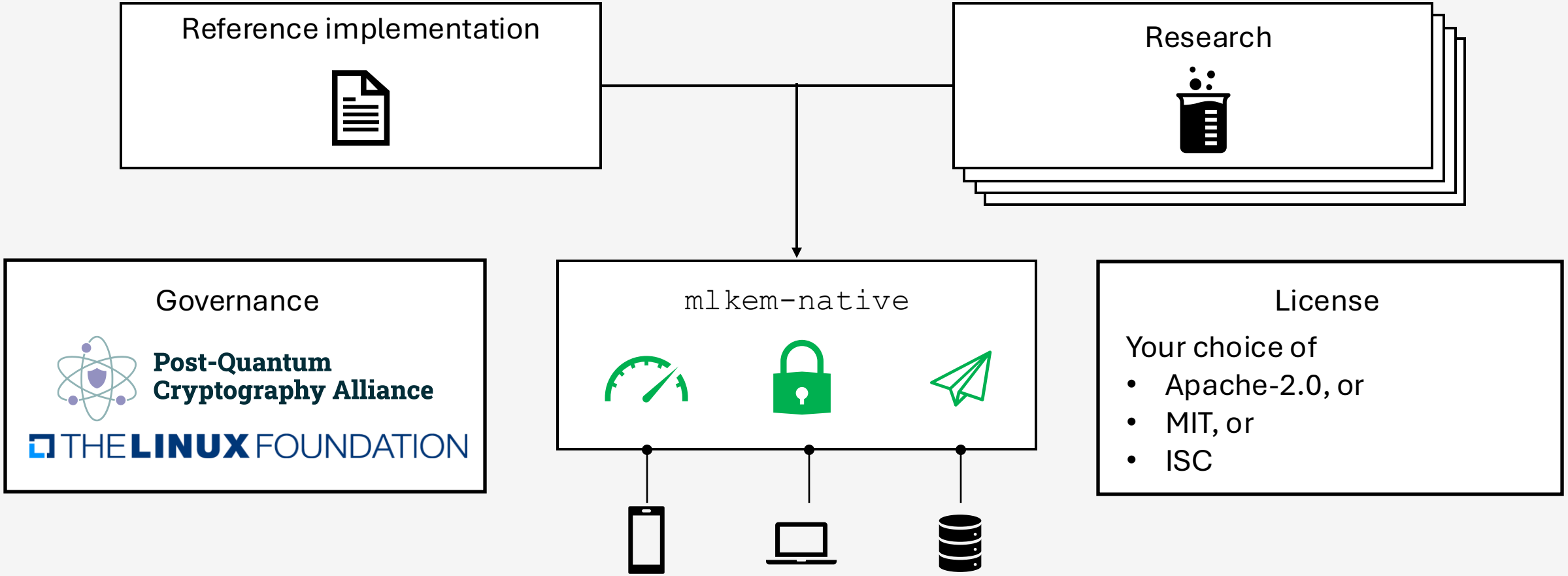
Why this talk is not interesting ...

- **ML-KEM:** Feels a bit '24. *Where's the novelty?*
- **C90:** Feels a bit.. well.. 90s. *Where's the memory safety?*
- **Formal Verification:** Slow, hard to maintain... overpromising?

Why this talk *is* interesting (?)

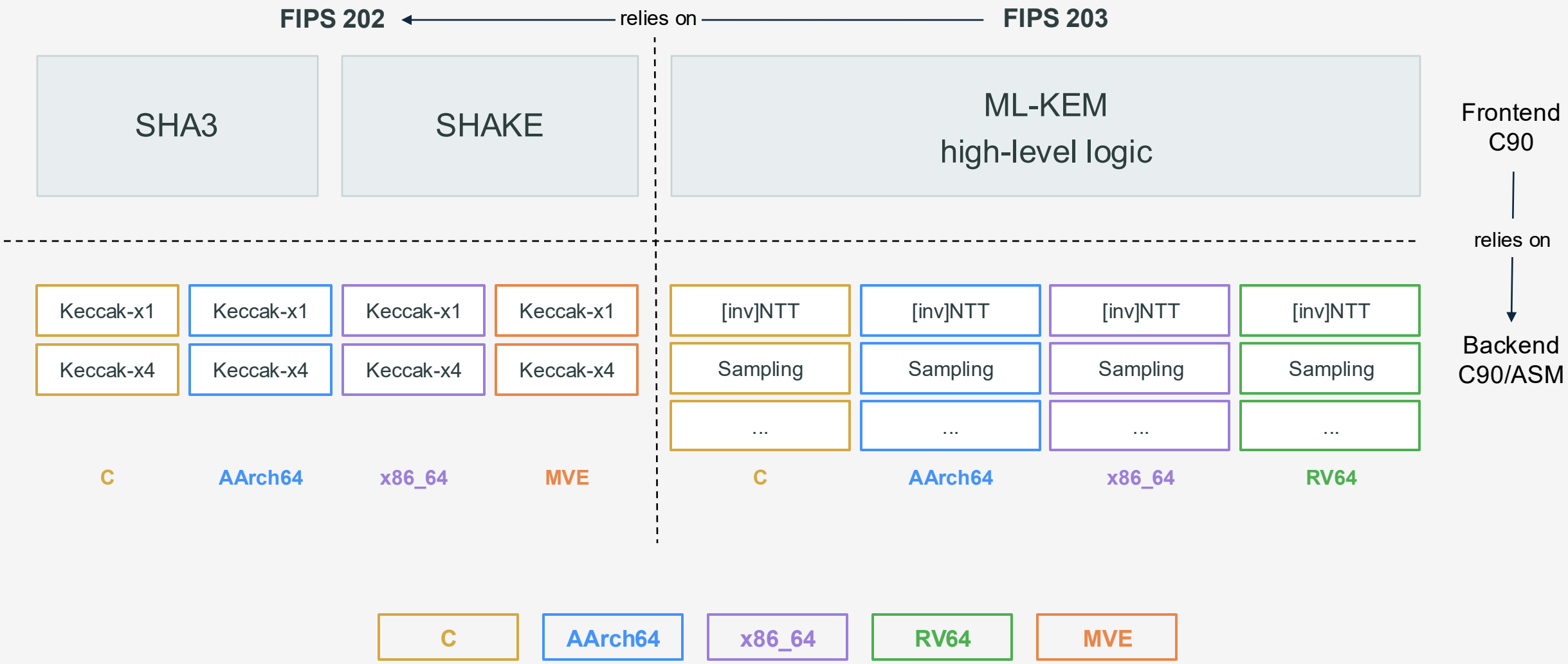
- **ML-KEM:** Case-study for more PQC to come.
- **C90:** Portable! Balance lack of safety through verification
- **Formal Methods:** Done right, **accelerates** code-delivery

mlkem-native: Idea

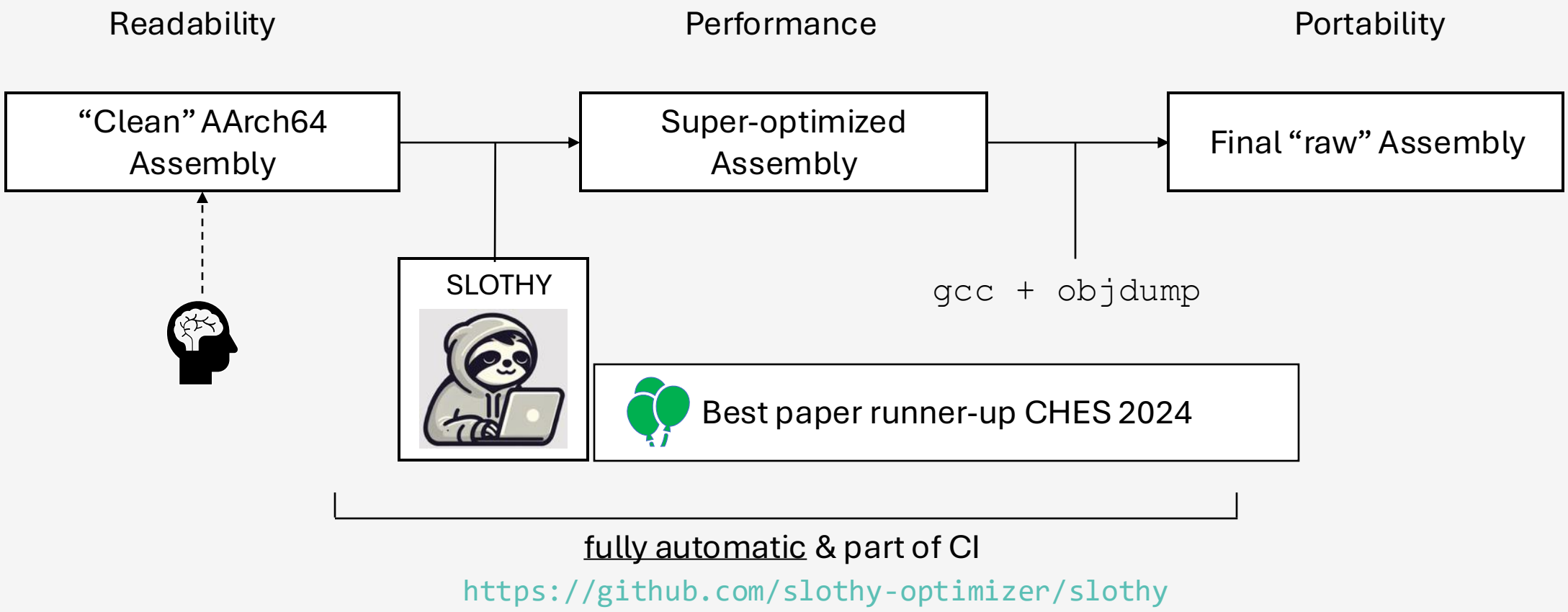


https://github.com/pq-code-package/mlkem-native

mlkem-native: Modularity



mlkem-native: Assembly optimization



Constant-Time Hardening

🚫 Secret-dependent memory access

🚫 Secret-dependent branches

🚫 Variable-time instructions

Assembly

What you write is what runs

We prove constant-time'ness in HOL Light (see later)

C Code

Different compilers/-O flags → different behavior

Compiler can optimize away constant-time patterns

KyberSlash, clangover, etc.

Approach: Value Barriers (Prevent reasoning about secret values)

```
// Approach 1: Inline assembly (gcc/clang)
static inline uint32_t mlk_value_barrier_u32(uint32_t b) {
    __asm__("" : "+r"(b)); return b;
}
```

```
// Approach 2: volatile opt-blocker (portable, Bernstein)
static inline uint32_t mlk_value_barrier_u32(uint32_t b) {
  return (b ^ (uint32_t)mlk_ct_opt_blocker_u64);
}
```

Formal Verification Overview

C

CBMC

SAT/SMT

Memory-Safety
Type-Safety

ASM

HOL Light + s2n-bignum

Interactive theorem proving

Functional Correctness
Memory Safety (- rejection sampling)
Constant-time

AArch64 | x86_64

- **Continuous Verification**: CBMC + HOL Light proofs run in CI
- **Reproducibility:** `nix` flake setting up entire toolchain – *try it!*

CBMC: Memory- and Type-Safety of C code

What?

Memory safety

No out-of-bounds memory accesses

Type safety

No signed/unsigned integer overflow

Other undefined behavior

- Undefined shift operations
- Division by zero

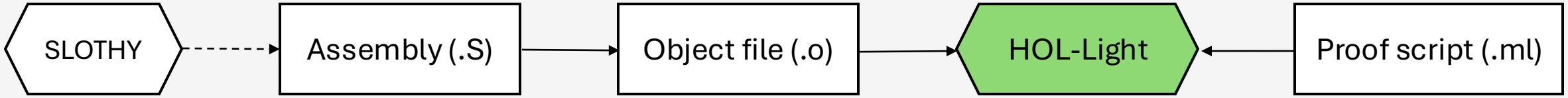
...

How?

Function contracts + loop invariants embedded directly in C source code.

```c
/* Reference: Embedded in `ntt()` in the reference implementation @[REF]. */
static void mlk_ntt_layer(int16_t r[MLKEM_N], unsigned layer)
__contract__(
  requires(memory_no_alias(r, sizeof(int16_t) * MLKEM_N))
  requires(1 <= layer && layer <= 7)
  requires(array_abs_bound(r, 0, MLKEM_N, layer * MLKEM_Q))
  assigns(memory_slice(r, sizeof(int16_t) * MLKEM_N))
  ensures(array_abs_bound(r, 0, MLKEM_N, (layer + 1) * MLKEM_Q)))
{
  unsigned start, k, len;
  /* Twiddle factors for layer n are at indices 2^(n-1)..2^n-1. */
  k = 1u << (layer - 1);
  len = (unsigned)MLKEM_N >> layer;
  for (start = 0; start < MLKEM_N; start += 2 * len)
  __loop__(
    invariant(start < MLKEM_N + 2 * len)
    invariant(k <= MLKEM_N / 2 && 2 * len * k == start + MLKEM_N)
    invariant(array_abs_bound(r, 0, start, layer * MLKEM_Q + MLKEM_Q))
    invariant(array_abs_bound(r, start, MLKEM_N, layer * MLKEM_Q)))
  {
    int16_t zeta = mlk_zetas[k++];
    mlk_ntt_butterfly_block(r, zeta, start, len, layer * MLKEM_Q);
  }
}
```

HOL Light Assembly Verification: Properties



"Post-hoc verification"

Functional Correctness

- Hoare-style pre/post-conditions
- *Program implicit*

"If, **initially**, the `PC` points to the object code of `PROG` and the system satisfies `PRE`, then **eventually**, the system will satisfy `POST`"

Typically:

```
PRE  := Memory(in_ptr) = data
POST := Memory(out_ptr) = f(data)
```

Constant-time & Memory-Safety

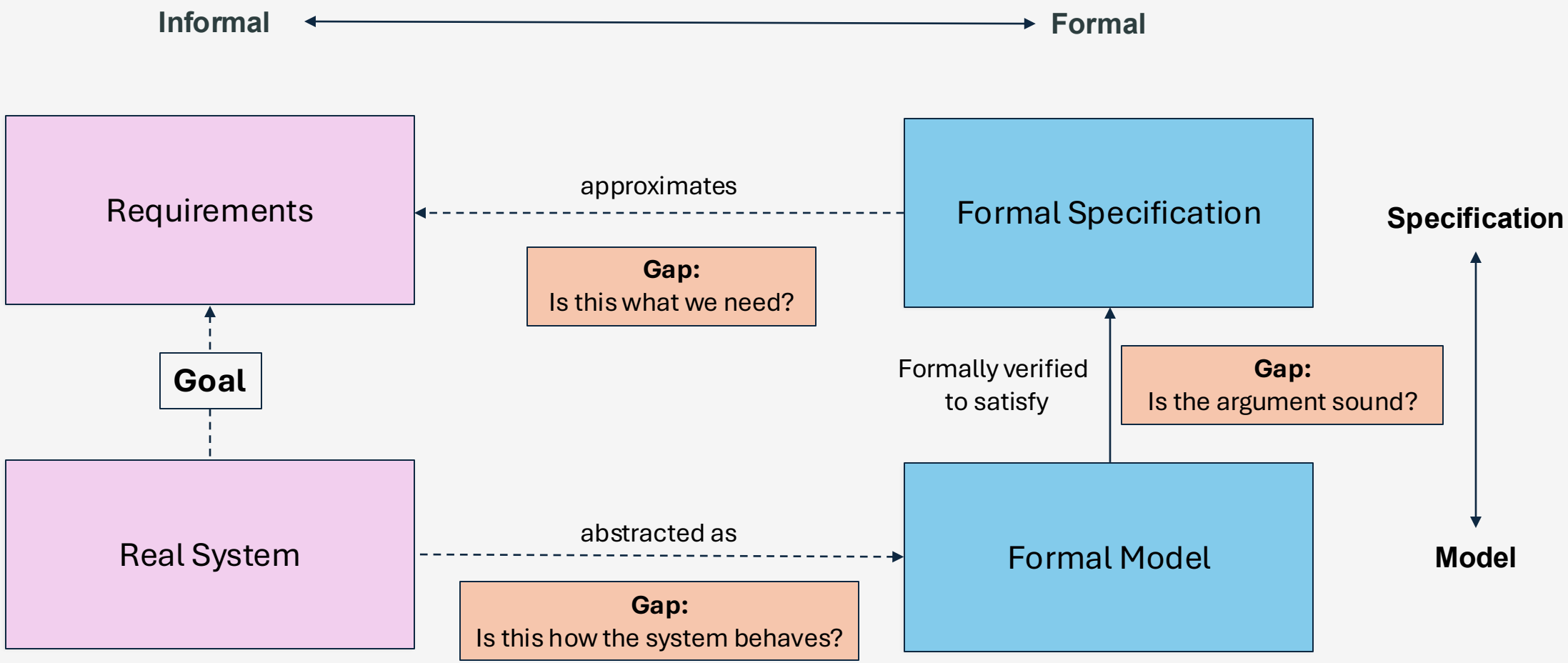
Idea: Programs yield traces of *events*:

- Branches (tagged with PC)
- Memory operations (tagged with address)
- Special variable-latency instructions

Prove: The event trace emitted…

- … is a function of <u>public</u> data
- … accesses memory within stated bounds

Formal verification: Gaps



SOUNDNESS.md

Formal verification is never absolute. Every verification effort links formal objects -- specifications and models -- to informal, real-world requirements and systems.
This document maps out what is proved about mlkem-native, **what is assumed, and where the gaps and risks lie.**
…
We see this as a living document. **If you have suggestions for improvements, such as soundness risks missing from or insufficiently covered in this document, please reach out** to us or open an issue.

mlkem-native/SOUNDNESS.md

Vision

- Should every formal verification project have a **SOUNDNESS.md**?
- Be open & transparent, invite discussion → build trust

mldsa-native: Doing It All Over Again

Same approach as mlkem-native – but for ML-DSA / FIPS 204

In Place

- CBMC proofs: frontend + C backend
- Constant-time hardening + testing
- AArch64 and x86_64 backend

Work in Progress

- HOL Light proofs for assembly (NTT/INTT done)
- Memory reduction
- SLOTHY optimization of AArch64 (NTT/INTTdone)

https://github.com/pq-code-package/mldsa-native

Why not use mlkem-native?

Portability & Maintainability

Assurance

Performance

Like mlkem-native, but it doesn’t fit your project yet?

Let us know!

Let’s **collaborate**, not duplicate!

Thank you!

https://github.com/pq-code-package/mlkem-native

https://github.com/pq-code-package/mldsa-native